Universidad del Valle de Guatemala

Facultad de Ingeniería

Reporte Escrito

MLflow en la plataforma Databricks

Machine Learning Engineering

Integrantes:

Andy Fuentes — 22944
Diederich Solís — 22952
Diego Linares — 221256
Christian Echeverria

Fecha: octubre de 2026

1 Introducción y objetivo

Se investigó el uso de MLflow en la plataforma Databricks y se adaptó el pipeline de scikit-learn del proyecto Champions League, desarrollado en el Taller 3 y la Actividad 9 [6]. El objetivo fue centralizar el seguimiento de entrenamiento, conservar modelos reutilizables y comprobar la misma lógica desde un Jupyter Notebook y un archivo Python.

La implementación se ejecutó en Databricks Free Edition con compute Serverless. Ambos formatos completaron la búsqueda de hiperparámetros de SVC, guardaron el modelo y verificaron su recarga. Este reporte presenta los resultados de esas ejecuciones, realizadas el 16 de octubre de 2026, hora de Guatemala.

1.1 MLflow y su integración con Databricks

MLflow organiza ejecuciones y empaqueta modelos. Databricks ofrece un servicio administrado de seguimiento y herramientas integradas para continuar su ciclo de vida [1]. En esta implementación, `uri="databricks"` dirige los registros al workspace y una ruta absoluta identifica el experimento [2].

Las funcionalidades aplicadas son: **Experiments**, para agrupar ejecuciones; **Tracking**, para conservar parámetros, métricas y artefactos; **Dataset Tracking**, para relacionar los datos con cada run; y **MLflow Models**, para guardar y cargar el pipeline completo [2, 5]. Model Registry en Unity Catalog, Feature Store y Model Serving forman parte del ecosistema, con configuración y objetivos específicos [1, 3, 4].

1.2 ¿Qué usos puede tener en proyectos de Machine Learning?

- Comparar configuraciones y modelos mediante métricas registradas.
- Mantener trazabilidad de los datos y parámetros usados en cada entrenamiento.
- Compartir resultados del equipo sin depender de salidas aisladas de notebooks.
- Recuperar el pipeline con sus transformaciones y firma para realizar inferencia.
- Preparar procesos de versionado y operación con servicios adicionales de la plataforma.

En Champions League, estas capacidades permiten evaluar variantes del clasificador y conservar evidencia de la configuración elegida. Las variables incluyen estadísticas del encuentro: el ejercicio clasifica resultados con datos observados durante el partido y no demuestra capacidad de pronóstico previo.

2 Cobertura de las etapas de CRISP-DM

La relación siguiente es el análisis metodológico del equipo. MLflow apoya de forma directa el modelado y la evaluación; el trabajo del equipo y las herramientas de datos de Databricks complementan las demás etapas.

Etapa	Aplicación en el proyecto	Apoyo
Comprensión del negocio	Definir la clasificación del resultado del partido y el criterio de éxito; documentar el propósito mediante notas y etiquetas.	Indirecto
Comprensión de datos	Identificar filas vacías, clases desbalanceadas, porcentajes, razones y columnas que revelan la respuesta. Relacionar datasets con los runs.	Parcial
Preparación de datos	Encapsular conversiones, imputación, escalado y codificación en el pipeline. Feature Store permite ampliar la reutilización de variables.	Parcial
Modelado	Calibrar SVC con RandomizedSearchCV y conservar parámetros de la búsqueda y del clasificador.	Directo
Evaluación	Registrar F1 macro de validación y prueba, exactitud y referencia; conservar artefactos y verificar la recarga.	Directo
Despliegue	Guardar el modelo reutilizable. Model Registry y Model Serving amplían el proceso hacia versiones gobernadas y endpoints.	Integración

2.1 Aplicación y alcance

El objetivo y la interpretación de resultados siguen siendo responsabilidad del equipo. En este laboratorio se completaron entrenamiento, evaluación, seguimiento y recuperación del modelo. El pipeline se guardó en MLflow, sin crear una versión en Unity Catalog ni un endpoint. La disponibilidad de un modelo guardado es un paso útil para su operación, pero no equivale por sí misma a un servicio desplegado.

3 Implementación del pipeline

3.1 Datos y preprocesamiento

El CSV contiene 151 filas iniciales. Tras retirar separadores vacíos quedan 144 partidos válidos. Se utilizaron 115 observaciones para entrenamiento y 29 para prueba, con partición estratificada y semilla 42. La variable objetivo representa victoria local, victoria visitante o empate.

Se excluyeron `score` y `winner` para evitar que la respuesta forme parte de las entradas. `ColumnTransformer` procesa grupos numéricos, porcentajes como texto, razones y equipos; aplica conversiones, imputación, escalado y One-Hot Encoding. El pipeline incorpora `SelectKBest` y el clasificador. El preprocesamiento se ajusta dentro de cada fold de la búsqueda.

3.2 Entrenamiento y configuración

Se reutilizó `entrenar_familia` del módulo de seguimiento del Taller 3. Se optimizó F1 macro mediante RandomizedSearchCV, con 40 configuraciones y cinco particiones de validación estratificada. La búsqueda se realiza sobre entrenamiento; el conjunto de prueba se conserva para evaluación.

```
URI = "databricks"
EXPERIMENTO = (
    "/Users/fue22944@uvg.edu.gt/"
    "Champions_Resultado_Databricks"
)
FAMILIA = "svc"
N_ITER = 40
FOLDS = 5
MODELO_UC = None
```

3.3 Notebook y archivo Python

El notebook importa `ejecutar` del archivo Python de la raiz. Para resolver los módulos propios se incorporaron a `sys.path` la raiz del repositorio y su carpeta `src`. Se instaló el proyecto en el entorno del notebook y se reinició Python antes de importar las bibliotecas.

El notebook ejecutó la función con `formato="notebook"`. Posteriormente se probó el archivo `.py` mediante `runpy.run_path(..., run_name="__main__")` y argumentos de línea de comandos, dentro del mismo entorno de Databricks. Esta segunda ejecución valida el punto de entrada del script; no corresponde a un Job independiente.

4 Resultados verificados en Databricks

Las dos ejecuciones completas utilizaron la misma configuración y partición. Las cifras siguientes proceden de las salidas reales del notebook y del archivo Python en Databricks, no de una nueva prueba local.

Métrica	**Notebook**	**Python**
F1 macro de validación cruzada	0.637570	0.637570
Desviación estándar de F1 en CV	0.051826	0.051826
Exactitud de prueba	0.655172	0.655172
F1 macro de prueba	0.601010	0.601010
F1 macro de referencia	0.217054	0.217054
Recarga del modelo verificada	Sí	Sí

4.1 Hiperparámetros seleccionados

- `C = 0.17535949529764422`.
- `class_weight = balanced` y `kernel = linear`.
- `gamma = 0.04609877941534893`; se conserva como parámetro de la búsqueda, aunque no interviene cuando el kernel de SVC es lineal.

4.2 Interpretación

La exactitud de prueba de 65.52% corresponde a 19 aciertos entre 29 partidos. El F1 macro de 0.601010 supera la referencia de clase mayoritaria de 0.217054. La igualdad entre ambos formatos muestra consistencia de la ejecución bajo la configuración utilizada; la recarga reprodujo el F1 macro de prueba.

La prueba inicial de dos iteraciones y tres folds obtuvo F1 macro de prueba de 0.646825. Esa cifra aislada no se utiliza para seleccionar el modelo: la búsqueda completa selecciona hiperparámetros por validación cruzada, y su esquema de validación es distinto. El conjunto de prueba pequeño limita la generalización de cualquier cifra individual.

Las cifras completas coinciden con la SVC reportada en el Taller 3. Las demás familias del taller son antecedentes y no se presentan como nuevas ejecuciones de esta entrega.

4.3 Estado del modelo

Ambos modelos quedaron guardados con una URI de MLflow Models y su recarga fue verificada. El valor `version=null` indica ausencia de una versión registrada en Model Registry; no significa que haya fallado el guardado. Se mantuvo `MODELO_UC=None`, sin registrar versiones ni alias en Unity Catalog y sin publicar Model Serving.

5 Trazabilidad y enlaces de entrega

5.1 Acceso al código y al notebook

Repositorio de GitHub:
https://github.com/Andyfer004/Ejercicio5-MLOPS
Notebook en el workspace de Databricks:
Abrir Champions_Databricks en Databricks
Notebook en GitHub:
notebooks/Champions_Databricks.ipynb
Archivo Python en GitHub:
pipeline_champions_databricks.py

5.2 Identificadores de las ejecuciones

Notebook:
Run: `bfe4871fb7324ad8953efb6bb7fa33cb`
Modelo: `models:/m-39d842fe62224cad9b83571b60c45968`
Abrir run del notebook
Archivo Python:
Run: `79e0d6ade66c4631b2cb47482a6b49f3`
Modelo: `models:/m-fa7ce8e782ae498fb1a3f6bdecababdb`
Abrir run del archivo Python

5.3 Registros y reproducibilidad

El código registra parámetros, métricas, datasets de entrenamiento y prueba, resultados de búsqueda, matriz de confusión, reporte de clasificación y el modelo con firma y código de transformadores. La comprobación de recarga genera `evaluacion/verificacion_recarga.json` y contrasta el F1 de prueba. Los enlaces de Databricks requieren los permisos correspondientes del workspace.

La implementación conserva la procedencia del pipeline y del dataset del Taller 3, publicado por Diego Linares, con autoría de los cuatro integrantes [6].

6 Evidencia de ejecución

6.1 Configuración completa

La captura muestra el servidor administrado de Databricks, la familia SVC, 40 iteraciones, cinco folds y el registro opcional de Unity Catalog desactivado.



Figura 1: Configuración ejecutada en Databricks. Captura proporcionada por el equipo.

6.2 Salida del notebook

La captura registra el run del notebook, el modelo guardado, los hiperparámetros y las métricas. El resultado completo de ambos formatos se resume en la sección 4 y sus identificadores se conservan en la sección 5.

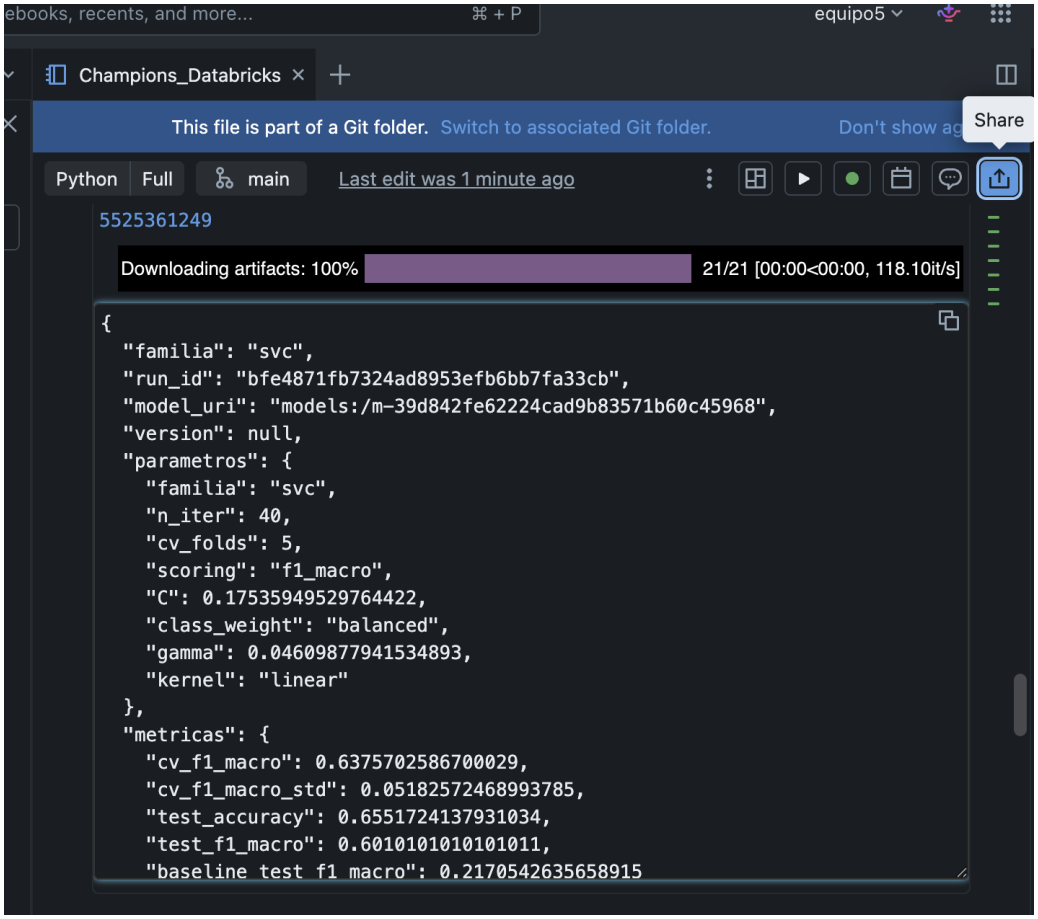


Figura 2: Salida de la búsqueda completa de SVC en Databricks. Captura proporcionada por el equipo.

7 Conclusiones

Se ejecutó la integración de scikit-learn y MLflow en Databricks tanto desde un notebook como desde un archivo Python. Ambos formatos obtuvieron el mismo F1 macro de validación de 0.637570 y F1 macro de prueba de 0.601010, con recarga verificada del modelo.

MLflow permitió vincular las ejecuciones con sus parámetros, métricas y modelos. La implementación conserva las transformaciones del pipeline y utiliza validación cruzada para seleccionar hiperparámetros, proporcionando trazabilidad y una forma consistente de recuperar el modelo.

Dentro de CRISP-DM, el aporte más directo se encuentra en modelado y evaluación. Databricks amplía el soporte a preparación, colaboración y operación mediante servicios complementarios. El alcance demostrado en este laboratorio es Experiments, Tracking, Dataset Tracking y Models.

8 Referencias

[1] Databricks. *MLflow on Databricks*.
https://docs.databricks.com/aws/en/mlflow/

[2] Databricks. *Track model development using MLflow*.
https://docs.databricks.com/aws/en/mlflow/tracking

[3] Databricks. *Manage model lifecycle in Unity Catalog*.
https://docs.databricks.com/aws/en/machine-learning/manage-model-lifecycle/

[4] Databricks. *Databricks Feature Store*.
https://docs.databricks.com/aws/en/machine-learning/feature-store/

[5] MLflow. *API mlflow.sklearn*.
https://mlflow.org/docs/latest/api_reference/python_api/mlflow.sklearn.html

[6] Equipo del curso. *Taller 3: MLflow con scikit-learn*. Octubre de 2026. Pipeline, dataset y reporte de referencia.
https://github.com/DiegoLinares11/MLFlow

[7] Equipo del curso. *Ejecuciones de Champions League en Databricks*. 16 de octubre de 2026. Salidas de notebook y archivo Python, capturas y enlaces del workspace incluidos en este informe.

Fecha de consulta: 16 de octubre de 2026.